K.Akshay
2303A51817

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name:B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
Instructor(s)Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju Mr. G. Kranthi		
Course Code	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/I	**Regulation**	R23
Date and Day of Assignment	Week 7 - Thursday	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All Batches
AssignmentNumber:14.4 (Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*ExpectedTime to complete*
1	**Lab 14: Web Frontend Development – AI-Assisted HTML/CSS/JS Generation**	Week 7

Lab Objectives

- Design **functional, visually appealing** web applications using HTML, CSS, and JavaScript with AI assistance.
- Apply **responsive, accessible, and interactive design principles**.
- Create **practical UI components** for real-world web applications.
- Use AI to optimize **layout, UX, and performance**.

Lab Outcomes

- Applying AI assistance to frontend development tasks
- Understanding how HTML, CSS, and JavaScript work together
- Improving responsiveness, interactivity, and usability
- Evaluating and refining AI-generated frontend code

Task 1: Responsive Homepage Layout for a Startup Website

Scenario

You are designing a **basic homepage for a startup company** that needs to look good on desktops, tablets, and mobile devices.

Task Description

- Create a web page structure using HTML that includes:
  - A header with navigation links
  - A main content section
  - A footer with basic information
- Use AI assistance to generate responsive CSS using media queries.
- Ensure the layout adjusts smoothly across different screen sizes.

Expected Output

- A responsive web page where:
  - Header, content, and footer are clearly separated
  - Layout adapts correctly to mobile, tablet, and desktop views
  - Code is clean and well-structured

Prompt:-

create a web page structure using Html which includes a header with navigationlinks with a main content section with a footer with basic information. also add css using media queries and ensure smooth layout handing

Code:-

lab 14.4 > index.html > html

```html
<html lang="en">
<head>
    <style>
        nav ul {
            list-style: none;
            display: flex;
            justify-content: center;
            flex-wrap: wrap;
        }

        nav ul li {
            margin: 0;
        }

        nav ul li a {
            display: block;
            color: white;
            text-decoration: none;
            padding: 1rem 1.5rem;
            transition: background-color 0.3s ease;
        }

        nav ul li a:hover {
            background-color: #2980b9;
        }

        main {
            max-width: 1200px;
            margin: 2rem auto;
            padding: 0 1rem;
            min-height: calc(100vh - 300px);
        }

        .content-section {
            background-color: #ecf0f1;
            padding: 2rem;
            margin-bottom: 2rem;
```

lab 14.4 > index.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Responsive Web Page</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: Arial, sans-serif;
            line-height: 1.6;
            color: #333;
        }

        header {
            background-color: #2c3e50;
            color: white;
            padding: 1rem 0;
            position: sticky;
            top: 0;
            z-index: 100;
        }

        header h1 {
            text-align: center;
            margin-bottom: 0.5rem;
        }

        nav {
            background-color: #34495e;
            padding: 0;
        }
```

lab 14.4 > index.html > html

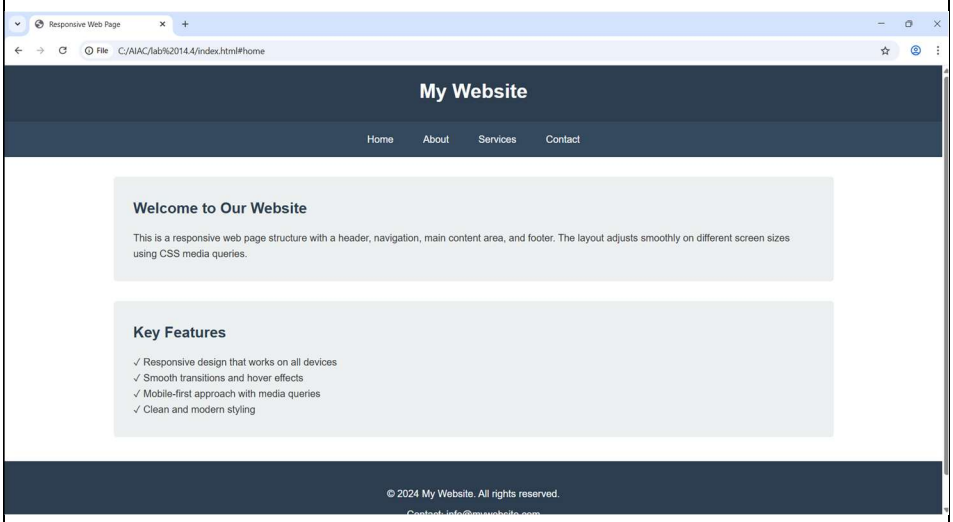
```html
<html lang="en">
<head>
    <style>
        .content-section h2 {
            color: #2c3e50;
            margin-bottom: 1rem;
        }

        footer {
            background-color: #2c3e50;
            color: white;
            text-align: center;
            padding: 2rem;
            margin-top: 2rem;
        }

        footer p {
            margin: 0.5rem 0;
        }

        /* Media Queries for Responsive Design */
        @media (max-width: 768px) {
            header h1 {
                font-size: 1.5rem;
            }

            nav ul {
                flex-direction: column;
            }

            nav ul li a {
                padding: 0.8rem 1rem;
                text-align: center;
            }

            main {
                margin: 1rem auto;
```

lab 14.4 > index.html > html

```html
<html lang="en">
<head>
    <style>

        @media (max-width: 480px) {
            header h1 {
                font-size: 1.2rem;
                padding: 0 0.5rem;
            }

            nav ul li a {
                padding: 0.6rem 0.8rem;
                font-size: 0.9rem;
            }

            main {
                padding: 0 0.5rem;
            }

            .content-section {
                padding: 1rem;
                margin-bottom: 1rem;
            }

            footer {
                padding: 1rem;
                font-size: 0.9rem;
            }
        }
    </style>
</head>
<body>
    <header>
        <h1>My Website</h1>
    </header>

    <nav>
```

```html
<html lang="en">
<body>
    <header>
    </header>

    <nav>
        <ul>
            <li><a href="#home">Home</a></li>
            <li><a href="#about">About</a></li>
            <li><a href="#services">Services</a></li>
            <li><a href="#contact">Contact</a></li>
        </ul>
    </nav>

    <main>
        <section class="content-section">
            <h2>Welcome to Our Website</h2>
            <p>This is a responsive web page structure with a header, navigation, main content area, and footer. The layout adjusts smoothly on
        </section>

        <section class="content-section">
            <h2>Key Features</h2>
            <p>✓ Responsive design that works on all devices</p>
            <p>✓ Smooth transitions and hover effects</p>
            <p>✓ Mobile-first approach with media queries</p>
            <p>✓ Clean and modern styling</p>
        </section>
    </main>

    <footer>
        <p>&copy; 2024 My Website. All rights reserved.</p>
        <p>Contact: info@mywebsite.com</p>
    </footer>
</body>
</html>
```

Output:-



Explanation: -

This task creates a basic webpage layout with a header, main content section, and footer using HTML. Responsive CSS with media queries is used so the layout adjusts properly for desktop, tablet, and mobile screens. This ensures the website looks good on different devices.

Task 2: Interactive Call-to-Action Button

Scenario

A marketing page requires a **call-to-action button** that responds when users interact with it.

Task Description

- Add a button to a web page using HTML.
- Use AI to generate JavaScript that responds to the button click by:
  - Displaying a message or alert
- Ask AI to add meaningful comments explaining the JavaScript logic.

Expected Output

- A web page where:
  - Clicking the button triggers a visible response (alert or message)
  - JavaScript code is readable and commented
  - No page reload occurs

Prompt:-

A marketing page requires a call-to-action button that responds when users interact with it. so add a button to that web page and generarte a javascript code that responds to that button click by displaying a message or an alert and also provide meaning ful comments explaining about the javascript

Code:-

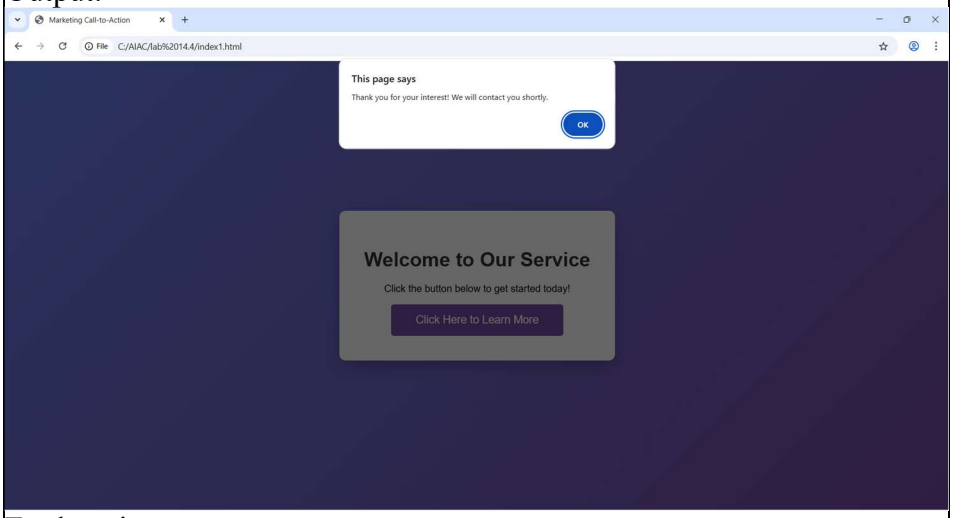
lab 14.4 > index1.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Marketing Call-to-Action</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            margin: 0;
        }
        .container {
            text-align: center;
            background: white;
            padding: 40px;
            border-radius: 10px;
            box-shadow: 0 10px 25px rgba(0, 0, 0, 0.2);
        }
        h1 {
            color: #333;
            margin-bottom: 20px;
        }
        .cta-button {
            background-color: #667eea;
            color: white;
            padding: 15px 40px;
            font-size: 18px;
            border: none;
            border-radius: 5px;
            cursor: pointer;
            transition: background-color 0.3s ease;
        }
```

lab 14.4 > index1.html > html

```html
<html lang="en">
<head>
    <style>
        .cta-button:hover {
            background-color: #764ba2;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>Welcome to Our Service</h1>
        <p>Click the button below to get started today!</p>
        <!-- Call-to-action button with id for JavaScript reference -->
        <button id="ctaBtn" class="cta-button">Click Here to Learn More</button>
    </div>

    <script>
        // Get reference to the button element using its id
        const button = document.getElementById('ctaBtn');

        // Add event listener to the button for 'click' events
        button.addEventListener('click', function() {
            // Display an alert message when button is clicked
            alert('Thank you for your interest! We will contact you shortly.');

            // Optional: You can also display a console message for debugging
            console.log('Call-to-Action button was clicked');
        });

        // Optional: Add hover effect feedback with console logging
        button.addEventListener('mouseenter', function() {
            console.log('User hovering over the button');
        });
    </script>
</body>
</html>
```

Output:-



Explanation:-

A call-to-action button is added using HTML, and JavaScript is used to detect when the user clicks the button. When clicked, it displays an alert or message without reloading the page. This makes the webpage interactive and improves user engagement.

Task 3: Contact Form with Client-Side Validation

Scenario

You are developing a **contact form for a company website** to collect user queries safely and correctly.

Task Description

- Design a form with fields for:
  - Name
  - Email
  - Message
- Use AI to generate JavaScript validation logic that:
  - Prevents empty submissions
  - Checks for a valid email format
- Display clear error messages for invalid input.

Expected Output

- A functional form where:
  - Invalid input shows inline error messages
  - Valid input displays a success message
  - Validation happens on the client side using JavaScript

Prompt:-

you should develop a contact form for a company website to collect user queries safely and correctly.Design a form with fields with Name,Email,Message .generate JavaScript validation logic does Prevents empty submissions and Checks for a valid email format . Display clear error messages for invalid input.

Code:-

lab 14.4 > index3.html > html

```html

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Contact Form</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: Arial, sans-serif;
            background-color: #f4f4f4;
            padding: 20px;
        }

        .container {
            max-width: 500px;
            margin: 50px auto;
            background-color: white;
            padding: 30px;
            border-radius: 8px;
            box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
        }

        h1 {
            color: #333;
            margin-bottom: 30px;
            text-align: center;
        }

        .form-group {
            margin-bottom: 20px;
        }

        label {
            display: block;
            margin-bottom: 8px;
            color: #333;
            font-weight: bold;
        }
```

gn 12.4.py | index.html | index2.html | index3.html × | add.py | assign-1.4.py | assign-3.4.py | assign-5.4.p

lab 14.4 > index3.html > html > head > style > body

```html
<html lang="en">
<head>
    <style>
        input:focus, textarea:focus {
            outline: none;
            border-color: #4CAF50;
            box-shadow: 0 0 5px rgba(76, 175, 80, 0.3);
        }

        textarea {
            resize: vertical;
            min-height: 120px;
        }

        .error {
            color: #d32f2f;
            font-size: 12px;
            margin-top: 5px;
            display: none;
        }

        .error.show {
            display: block;
        }

        input.invalid, textarea.invalid {
            border-color: #d32f2f;
            background-color: #ffebee;
        }

        button {
            width: 100%;
            padding: 12px;
            background-color: #4CAF50;
            color: white;
            border: none;
            border-radius: 4px;
            font-size: 16px;
            font-weight: bold;
            cursor: pointer;
            transition: background-color 0.3s;
        }

        button:hover {
            background-color: #45a049;
```

lab 14.4 > index3.html > html > head > style > body

```html
<html lang="en">
<body>
    <div class="container">
        <h1>Contact Us</h1>
        <form id="contactForm" novalidate>
            <div class="form-group">
                <label for="name">Name</label>
                <input type="text" id="name" name="name" placeholder="Enter your full name">
                <div class="error" id="nameError"></div>
            </div>

            <div class="form-group">
                <label for="email">Email</label>
                <input type="email" id="email" name="email" placeholder="Enter your email address">
                <div class="error" id="emailError"></div>
            </div>

            <div class="form-group">
                <label for="message">Message</label>
                <textarea id="message" name="message" placeholder="Enter your message"></textarea>
                <div class="error" id="messageError"></div>
            </div>

            <button type="submit">Send Message</button>
        </form>
        <div class="success" id="successMessage">✓ Your message has been sent successfully!</div>
    </div>

    <script>
        const form = document.getElementById('contactForm');
        const nameInput = document.getElementById('name');
        const emailInput = document.getElementById('email');
        const messageInput = document.getElementById('message');
        const successMessage = document.getElementById('successMessage');

        // Validation rules
        const validate = () => {
            let isValid = true;

            // Validate Name
            if (!nameInput.value.trim()) {
                showError('nameError', 'Name is required');
                nameInput.classList.add('invalid');
                isValid = false;
            } else if (nameInput.value.trim().length < 2) {
```

gn 12.4.py | index.html | index2.html | index3.html | add.py | assign-1.4.py | assign-3.4.py | assig

lab 14.4 > index3.html > html > head > style > body

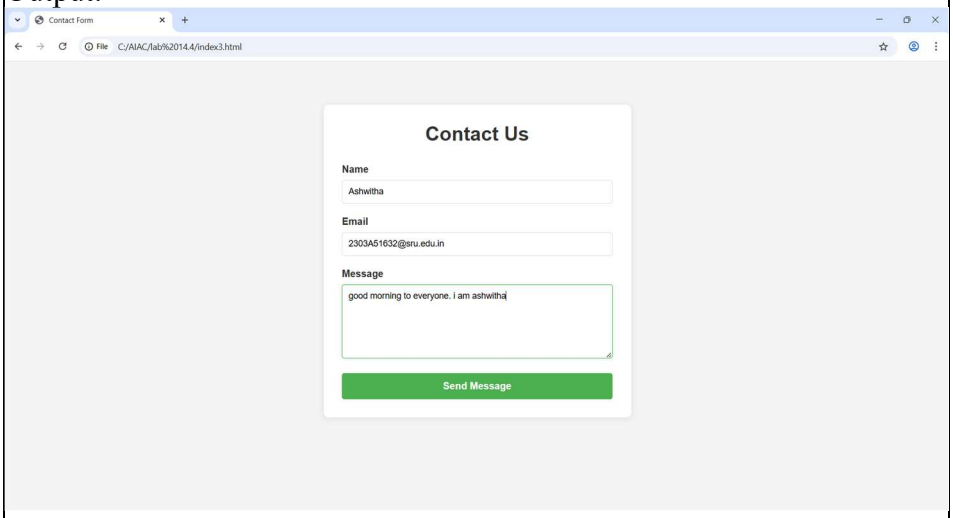
```html
<html lang="en">
<body>
    <script>
        const validate = () => {

            // Validate Email
            const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
            if (!emailInput.value.trim()) {
                showError('emailError', 'Email is required');
                emailInput.classList.add('invalid');
                isValid = false;
            } else if (!emailRegex.test(emailInput.value.trim())) {
                showError('emailError', 'Please enter a valid email address');
                emailInput.classList.add('invalid');
                isValid = false;
            } else {
                clearError('emailError');
                emailInput.classList.remove('invalid');
            }

            // Validate Message
            if (!messageInput.value.trim()) {
                showError('messageError', 'Message is required');
                messageInput.classList.add('invalid');
                isValid = false;
            } else if (messageInput.value.trim().length < 10) {
                showError('messageError', 'Message must be at least 10 characters long');
                messageInput.classList.add('invalid');
                isValid = false;
            } else {
                clearError('messageError');
                messageInput.classList.remove('invalid');
            }

            return isValid;
        };

        // Show error message
        const showError = (errorId, message) => {
            const errorElement = document.getElementById(errorId);
            errorElement.textContent = message;
            errorElement.classList.add('show');
        };

        // Clear error message
```

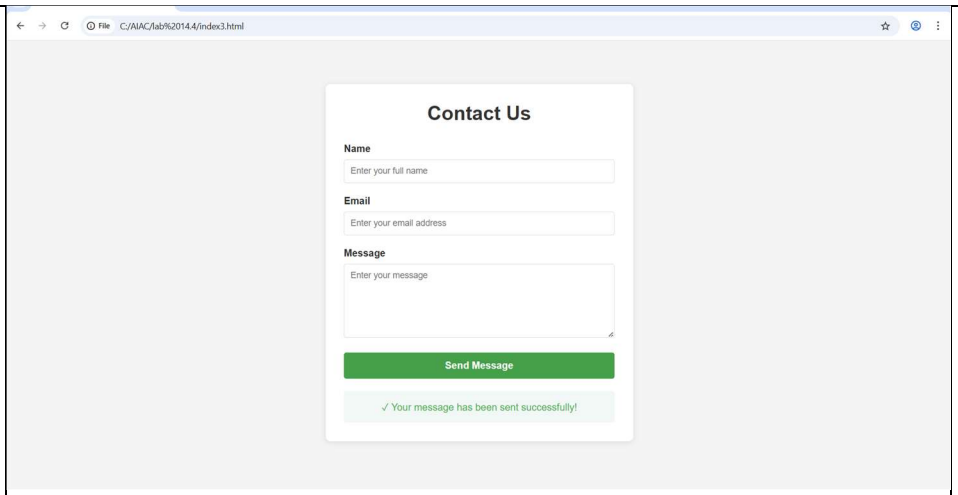
lab 14.4 > index3.html > html > body > script > validate

```html
<html lang="en">
<body>
    <script>
        const showError = (errorId, message) => {
        };

        // Clear error message
        const clearError = (errorId) => {
            const errorElement = document.getElementById(errorId);
            errorElement.textContent = '';
            errorElement.classList.remove('show');
        };

        // Clear errors on input
        nameInput.addEventListener('input', () => clearError('nameError'));
        emailInput.addEventListener('input', () => clearError('emailError'));
        messageInput.addEventListener('input', () => clearError('messageError'));

        // Form submission
        form.addEventListener('submit', (e) => {
            e.preventDefault();
            cessMessage.classList.remove('show');

            if (validate()) {
                // Simulate sending data
                console.log({
                    name: nameInput.value.trim(),
                    email: emailInput.value.trim(),
                    message: messageInput.value.trim()
                });

                // Reset form and show success message
                form.reset();
                successMessage.classList.add('show');

                // Hide success message after 5 seconds
                setTimeout(() => {
                    successMessage.classList.remove('show');
                }, 5000);
            }
        });
    </script>
</body>
</html>
```

Click to add a breakpoint

Output:-





Explanation:-
A contact form is created with fields for name, email, and message. JavaScript validation checks that all fields are filled and the email format is correct before submission. Error messages are shown for invalid input, and a success message appears for valid entries.

Task 4: Dynamic List Management for a Product Page

Scenario

A product listing page needs to **add or remove items dynamically** without refreshing the page.

Task Description

- Create a simple HTML list to display items.
- Use AI to generate JavaScript that:
  - Adds a new item when an “Add” button is clicked
  - Removes the last item when a “Remove” button is clicked
- Ensure the DOM updates dynamically.

Expected Output

- A web page where:
  - Items can be added and removed dynamically
  - Changes appear instantly without page reload
  - JavaScript logic is clear and maintainable

Prompt:-

A product listing page needs to add or remove items dynamically without refreshing the page. Create a simple HTML list to display items.Use AI to generate JavaScript that:Adds a new item when an "Add" button is clicked and it should remove the last item when a "Remove" button is clicked Ensure the DOM updates dynamically.

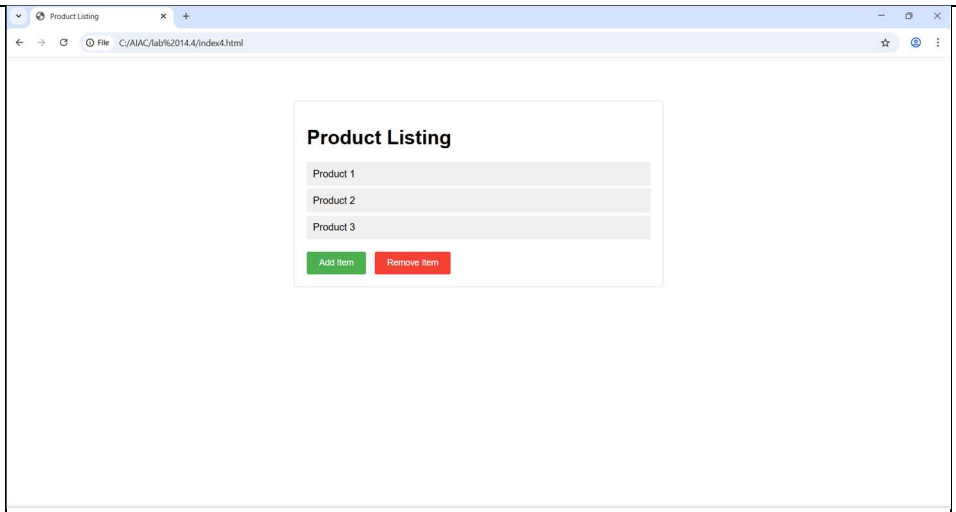
Code:-

lab 14.4 > index4.html > html

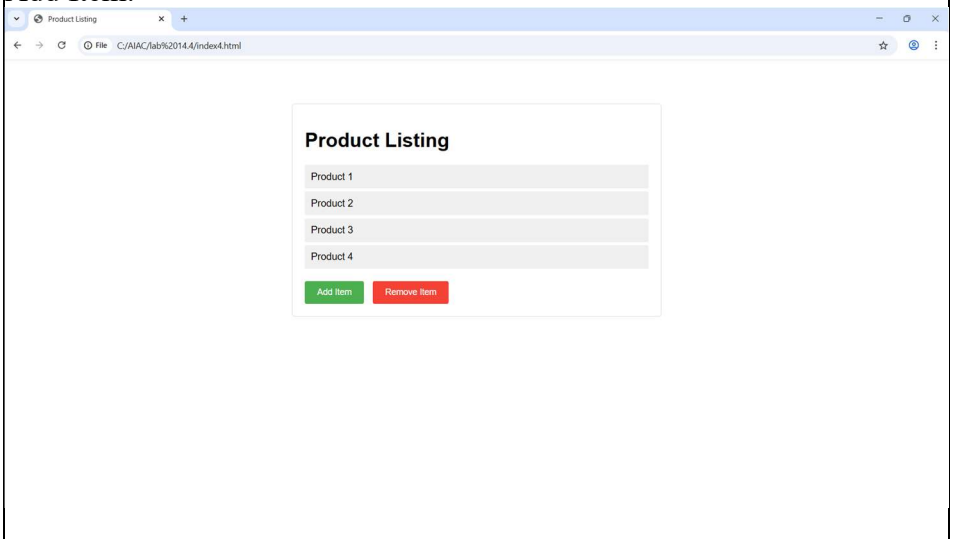
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Product Listing</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            max-width: 600px;
            margin: 50px auto;
            padding: 20px;
        }
        .container {
            border: 1px solid #ddd;
            padding: 20px;
            border-radius: 5px;
        }
        #productList {
            list-style-type: none;
            padding: 0;
        }
        #productList li {
            padding: 10px;
            margin: 5px 0;
            background-color: #f0f0f0;
            border-radius: 3px;
        }
        .button-group {
            margin-top: 20px;
        }
        button {
            padding: 10px 20px;
            margin-right: 10px;
            border: none;
            border-radius: 3px;
            cursor: pointer;
            font-size: 14px;
        }
        #addBtn {
            background-color: #4CAF50;
            color: white;
        }
```

```html
<html lang="en">
<head>
    <style>
        #removeBtn {
            background-color: #f44336;
            color: white;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>Product Listing</h1>
        <ul id="productList">
            <li>Product 1</li>
            <li>Product 2</li>
            <li>Product 3</li>
        </ul>
        <div class="button-group">
            <button id="addBtn">Add Item</button>
            <button id="removeBtn">Remove Item</button>
        </div>
    </div>

    <script>
        const productList = document.getElementById('productList');
        const addBtn = document.getElementById('addBtn');
        const removeBtn = document.getElementById('removeBtn');
        let itemCount = 3;

        addBtn.addEventListener('click', function() {
            itemCount++;
            const newItem = document.createElement('li');
            newItem.textContent = `Product ${itemCount}`;
            productList.appendChild(newItem);
        });

        removeBtn.addEventListener('click', function() {
            if (productList.children.length > 0) {
                productList.removeChild(productList.lastChild);
            }
        });
    </script>
</body>
</html>
```

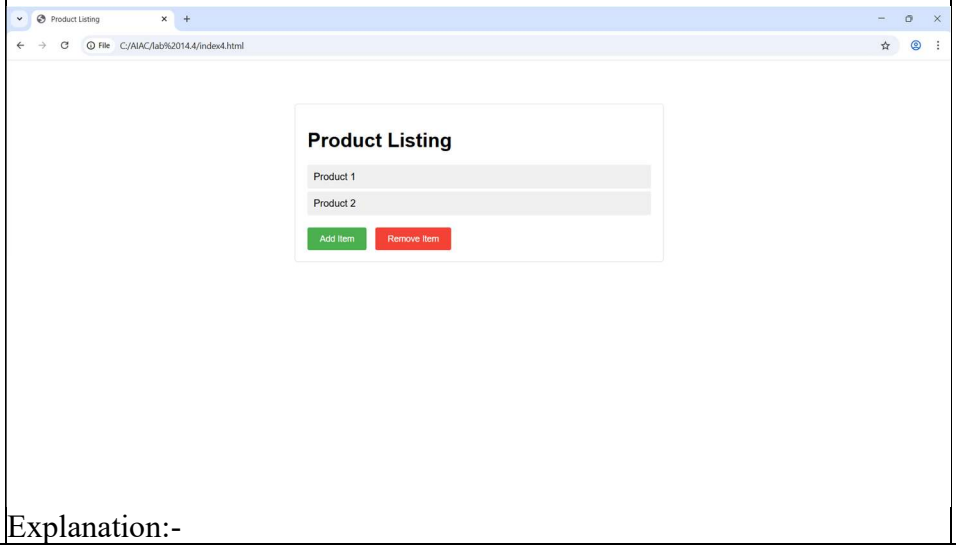
Output:-



Add Item:-



Remove item:-



Explanation:-

An HTML list is used to display items, and JavaScript allows items to be added or removed dynamically. When the “Add” button is clicked, a new item is added to the list, and the “Remove” button deletes the last item. The page updates instantly without reloading.

Task 5: Modal Popup for User Notifications

Scenario

You are building a **user notification popup** for announcements or alerts on a website.

Task Description

- Use AI to help generate:
  - HTML structure for a modal popup
  - CSS styling with a semi-transparent background overlay
  - JavaScript to open and close the modal
- Include multiple ways to close the modal (button or overlay click).

Expected Output

- A web page where:
  - Clicking “Open Modal” displays the popup
  - Popup overlays the page content clearly
  - Modal closes smoothly when instructed

Prompt:-

need to build a user notification popup for announcements or alerts on a website. in that website html structure for a modal pop up and use CSS styling with a semi-transparent background overlay and JavaScript to open and close the modal include multiple ways to close the modal

Code:-

index3.html | index4.html | Untitled-1 | index5.html | add.py | assign-1.4.py | assign-3.4.py

lab 14.4 > index5.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Notification Modal Popup</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: Arial, sans-serif;
            background-color: #f5f5f5;
            padding: 20px;
        }

        button {
            padding: 10px 20px;
            background-color: #007bff;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
            font-size: 16px;
        }

        button:hover {
            background-color: #0056b3;
        }

        /* Modal Overlay */
        .modal-overlay {
            display: none;
            position: fixed;
            top: 0;
            left: 0;
            width: 100%;
            height: 100%;
            background-color: rgba(0, 0, 0, 0.5);
            z-index: 999;
        }

```

index3.html | index4.html | Untitled-1 | index5.html | add.py | assign-1.4.py | assign-3.4.py

lab 14.4 > index5.html > html

```html
<html lang="en">
<head>
    <style>
        .modal-overlay.active {
            display: flex;
            justify-content: center;
            align-items: center;
        }

        /* Modal Container */
        .modal {
            background-color: white;
            padding: 30px;
            border-radius: 10px;
            box-shadow: 0 4px 6px rgba(0, 0, 0, 0.2);
            max-width: 500px;
            width: 90%;
            position: relative;
            animation: slideIn 0.3s ease-out;
        }

        @keyframes slideIn {
            from {
                transform: translateY(-50px);
                opacity: 0;
            }
            to {
                transform: translateY(0);
                opacity: 1;
            }
        }

        /* Close Button (X) */
        .modal-close-btn {
            position: absolute;
            top: 10px;
            right: 15px;
            background: none;
            border: none;
            font-size: 28px;
            color: #999;
            cursor: pointer;
            padding: 0;
            width: 30px;
            height: 30px;
```

lab 14.4 > index5.html > html

```
<html lang="en">
<head>
    <style>
        .modal-close-btn {
            display: flex;
            justify-content: center;
            align-items: center;
        }

        .modal-close-btn:hover {
            color: #333;
            background-color: #f0f0f0;
            border-radius: 50%;
        }

        .modal-header {
            margin-bottom: 15px;
            padding-right: 30px;
        }

        .modal-header h2 {
            color: #333;
            margin-bottom: 5px;
        }

        .modal-body {
            color: #666;
            line-height: 1.6;
            margin-bottom: 20px;
        }

        .modal-footer {
            display: flex;
            gap: 10px;
            justify-content: flex-end;
        }

        .btn-close {
            background-color: #6c757d;
        }

        .btn-close:hover {
            background-color: #5a6268;
        }

```

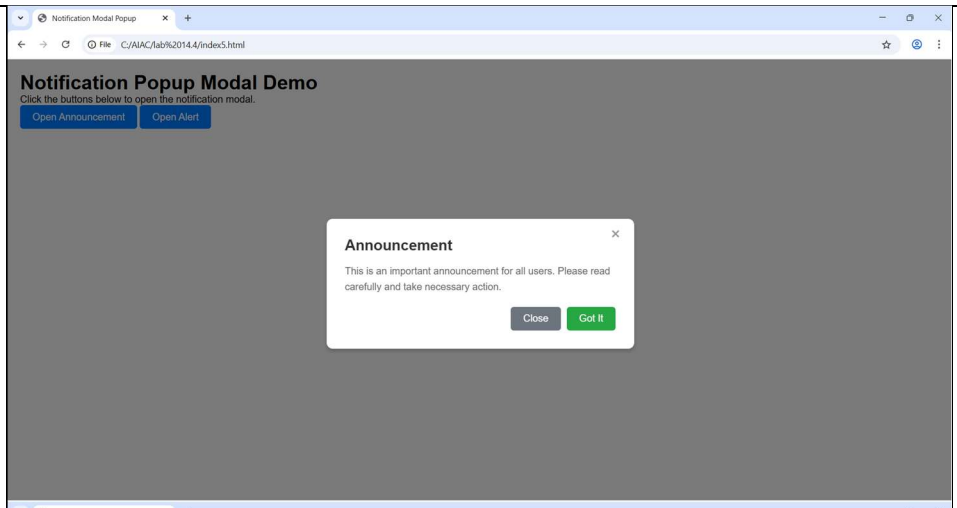
lab 14.4 > index5.html > html

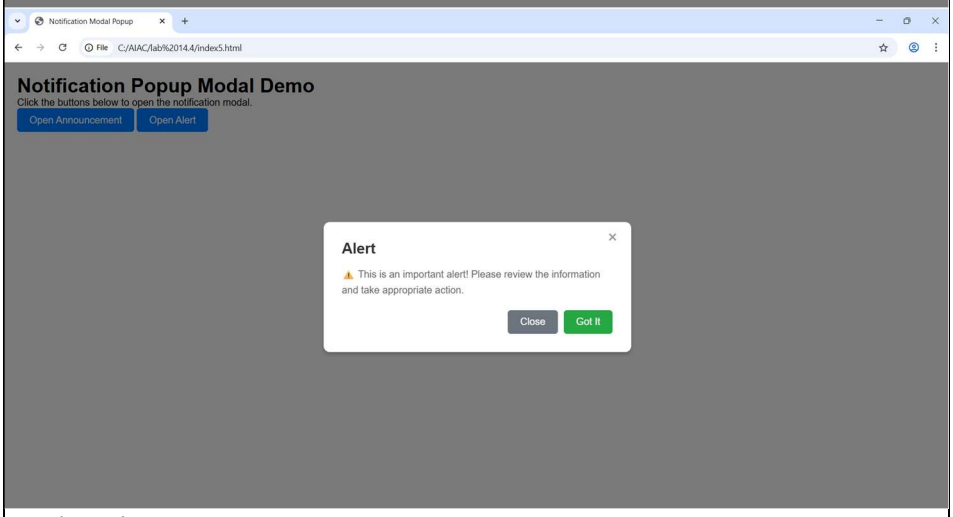
```html
<html lang="en">
<head>
    <style>
        .btn-close:hover {
            background-color: #5a6268;
        }

        .btn-primary {
            background-color: #28a745;
        }

        .btn-primary:hover {
            background-color: #218838;
        }
    </style>
</head>
<body>
    <h1>Notification Popup Modal Demo</h1>
    <p>Click the buttons below to open the notification modal.</p>

    <button onclick="openModal()">Open Announcement</button>
    <button onclick="openAlertModal()">Open Alert</button>

    <!-- Modal Overlay -->
    <div class="modal-overlay" id="modalOverlay">
        <!-- Modal Container -->
        <div class="modal" id="modal">
            <!-- Close Button (X) -->
            <button class="modal-close-btn" onclick="closeModal()">&times;</button>

            <!-- Modal Header -->
            <div class="modal-header">
                <h2 id="modalTitle">Announcement</h2>
            </div>

            <!-- Modal Body -->
            <div class="modal-body" id="modalBody">
                <p>This is an important notification for you. You can close this modal in multiple ways.</p>
            </div>

            <!-- Modal Footer -->
            <div class="modal-footer">
                <button class="btn-close" onclick="closeModal()">Close</button>
                <button class="btn-primary" onclick="closeModal()">Got It</button>
            </div>
```

lab 14.4 > index5.html > html

```html
<html lang="en">
<body>
    <script>
        const modalOverlay = document.getElementById('modalOverlay');
        const modalTitle = document.getElementById('modalTitle');
        const modalBody = document.getElementById('modalBody');

        // Open Modal
        function openModal() {
            modalTitle.textContent = 'Announcement';
            modalBody.innerHTML = '<p>This is an important announcement for all users. Please read carefully and take necessary action.</p>';
            modalOverlay.classList.add('active');
        }

        // Open Alert Modal
        function openAlertModal() {
            modalTitle.textContent = 'Alert';
            modalBody.innerHTML = '<p>⚠️ This is an important alert! Please review the information and take appropriate action.</p>';
            modalOverlay.classList.add('active');
        }

        // Close Modal
        function closeModal() {
            modalOverlay.classList.remove('active');
        }

        // Close Modal by clicking outside (on overlay)
        modalOverlay.addEventListener('click', (event) => {
            if (event.target === modalOverlay) {
                closeModal();
            }
        });

        // Close Modal on Escape key
        document.addEventListener('keydown', (event) => {
            if (event.key === 'Escape' && modalOverlay.classList.contains('active')) {
                closeModal();
            }
        });
    </script>
</body>
</html>
```

Output:-





Explanation:-
A modal popup is created using HTML, CSS, and JavaScript to display notifications. Clicking the “Open Modal” button shows the popup with a background overlay. The modal can be closed using a close button or by clicking outside the popup.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots